Project Review

Adding Dimensions | Teacher / Student Training Tracker

Technical review, demo access, database handover and local installation guide

Prepared: 27 October 2026 | Source folder: D:\SchoolNet Projects\Adding Dimensions

Scope: the current local React, Ocelot, .NET 10 and PostgreSQL implementation. This document records verified behavior and identifies remaining production/BRD work; it is not a full business-acceptance sign-off.

Handover item	Location / meaning
Project Review.docx	Editable Word review in the Adding Dimensions root.
Project Review.pdf	Portable reading/printing copy of this review.
Database Import/AddingDimensions.backup	Recommended PostgreSQL custom-format database archive.
Database Import/AddingDimensions.sql	Plain SQL alternative for psql imports.
Database Import/setup_local.py	Creates a fresh database/login, restores the archive, copies learning files and configures both .NET projects.
Database Import/application-encryption.private.json	Private keys needed to read the existing encrypted contacts and retain contact matching.
Adding Dimensions Database Import.zip	Convenience archive of the complete Database Import folder; source code is copied separately.

Contents

1. Project purpose and present status

2. Major folders and handover layout

3. Architecture and request flow

4. Technology and software prerequisites

5. Front-End CodeBase review

6. Back-End CodeBase review

7. PostgreSQL design and sample data

8. Reference Documents review

9. Demo credentials and role access

10. Login, OTP, registration and recovery flows

11. New-machine setup: recommended method

12. Manual PostgreSQL import and pgAdmin guidance

13. Daily running, configuration and development

14. Validation and test evidence

15. Troubleshooting

16. Remaining work and deployment boundaries

Appendix A. Database table inventory

Appendix B. Views and stored procedures

Appendix C. Frontend collections and API routes

Appendix D. Source and download references

1. Project purpose and present status

Adding Dimensions is a role-based training-management application for sponsors, Schoolnet administrators, geographic/school administrators, trainers, trainees and reporting users. It brings project planning, reusable training plans, batch scheduling, participant registration, attendance, learning material, assessments, feedback, certificates and reporting into one local application.

The frontend was supplied through the TeacherTraining GitHub repository. The backend adapts the NISM Sync MicroServices coding conventions to .NET 10 and PostgreSQL, using the client workbook and BRD as references. The NISM reference project is not a runtime dependency on another machine.

- Operational records are read from and saved to PostgreSQL through the local API gateway; dashboards and report rows use saved records.
- The frontend layout, styling, assets and folder structure were retained. API/data handlers and OTP controls were integrated into the existing screens.
- All eight existing demo IDs and their shared demo password are preserved. Login now requires password verification followed by OTP verification.
- OTP uses a local email outbox, as selected for this stage. No external email/SMS delivery service is configured.
- The application is a local integrated build. Live service integrations, additional approval/privacy workflows and production hosting are not claimed complete.

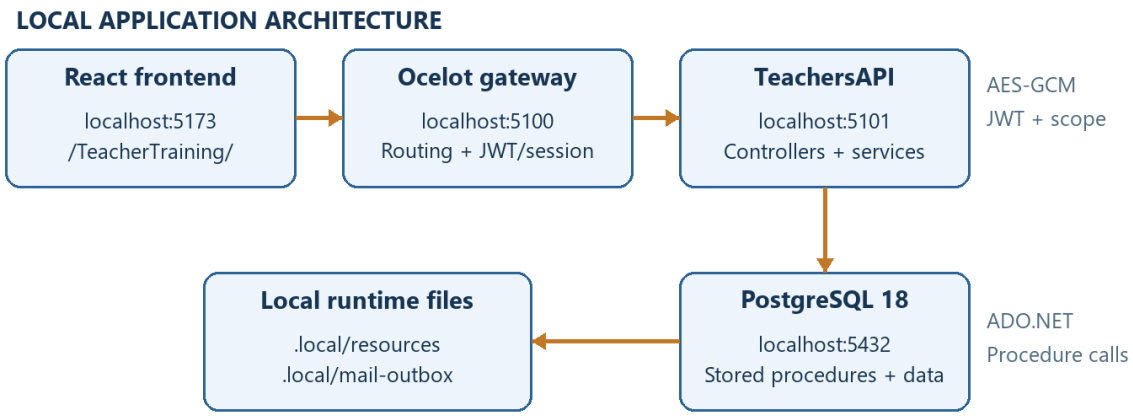
2. Major folders and handover layout

```
Adding Dimensions/
  Front-End CodeBase/
  Back-End CodeBase/
  Reference Documents/
  Database Import/
  Project Review.docx
  Project Review.pdf
  Adding Dimensions Database Import.zip
```

Folder	Responsibility	Needed to run?
Front-End CodeBase	React/TypeScript screens, routing, styles, resources, browser API adapter, build/deployment definitions and user/test documentation.	Yes
Back-End CodeBase	TeachersAPI, JWT library, Ocelot gateway, SQL definitions, launch scripts, setup helpers and automated tests.	Yes
Reference Documents	Client database workbook, business requirements, clarifications and dashboard reference image.	For design/maintenance; not a runtime dependency
Database Import	Database snapshot, private PII keys, learning files, setup helper, inventory/checksums and restore verification.	Needed for this snapshot-based installation

On another Windows machine, these folder names should remain siblings. The parent directory can be on a different drive, for example C:\Projects\Adding Dimensions. The launch script finds the frontend relative to the backend; it does not require the original D: drive.

3. Architecture and request flow



The browser uses http://localhost:5173/TeacherTraining/. API requests go to Ocelot on port 5100, which forwards to TeachersAPI on port 5101. TeachersAPI calls PostgreSQL through Npgsql/ADO.NET. PostgreSQL normally listens on port 5432.

- src/api.ts encrypts JSON requests with AES-256-GCM and decrypts the response envelope. The reference-compatible key derivation uses PBKDF2-SHA256, 1,000 iterations, a 16-byte salt, 12-byte nonce and 16-byte tag.
- Controllers decrypt and validate requests, then call service classes. Services map screen contracts into typed parameters and relational fields.
- SqlHelper executes named PostgreSQL procedures with CommandType.StoredProcedure and converts their INOUT JSONB results to DataTable. There are no inline runtime SELECT/INSERT/UPDATE/DELETE queries in C#.
- Services return the reference-style Message<T> result: success flag, HTTP/status code, return message, data/data list and request/response metadata. Ordinary application responses are encrypted; health and file-download routes have their appropriate plain/binary formats.
- JWTs are issued only after OTP verification. Sessions are stored in the database and checked at the gateway/API; logout and a newer login invalidate the previous session.
- Contact encryption at rest is separate from transport encryption. Existing database records require the exported Pii.EncryptionKey and Pii.BlindIndexKey. The import helper preserves those keys but generates fresh JWT and transport secrets.
- Learning-file bytes are outside PostgreSQL in Back-End CodeBase/.local/resources. Database metadata points to them, and signed download links enforce session/scope/expiry.

4. Technology and software prerequisites

Component	Verified project version / requirement	Purpose
Windows / PowerShell	Windows with Windows PowerShell 5.1+ or compatible PowerShell; launcher uses Windows APIs.	Documented automatic launch/import workflow
.NET SDK	10.0.403; global.json uses latestPatch within its feature band. Install the pinned SDK.	Build all three .NET projects
.NET runtime / JWT bearer	10.0.14 / net10.0	API and authentication runtime
PostgreSQL	18.6 tested; use PostgreSQL 18 client tools for this dump.	Database, psql, pg_dump, pg_restore, pg_ctl
Node.js / npm	24.22.1 / 11.19.1 tested	Frontend installation/build/development server
React / TypeScript / Vite	19.3.1 / 5.9.3 / 7.3.7	UI and frontend tooling
Material UI / Redux Toolkit	7.3.11 / 2.13.0	Existing UI components and presentation state

Component	Verified project version / requirement	Purpose
Npgsql / Ocelot	10.0.3 / 25.0.2	ADO.NET database provider and API gateway
Newtonsoft.Json / Swagger	13.0.4 / Swashbuckle 10.3.1	JSON compatibility and local API documentation
Python	3.11+ for import/helper scripts; 3.15.2 used for verification. No pip packages needed for setup_local.py.	Import/provisioning and maintenance helpers
Chrome / Git / IDE	Chrome for UI checks; Git optional for cloning; VS Code or Visual Studio optional.	Browser and developer convenience

Install the SDK rather than only the .NET runtime. Installing an arbitrary earlier .NET 10 feature band may not satisfy global.json. The published .NET and PostgreSQL download links are listed in Appendix D. Do not upgrade application dependencies merely to perform the handover; npm ci uses the supplied lockfile.

```
dotnet --list-sdks
dotnet --version
node --version
npm.cmd --version
python --version
& 'C:\Program Files\PostgreSQL\18\bin\psql.exe' --version
& 'C:\Program Files\PostgreSQL\18\bin\pg_restore.exe' --version
```

5. Front-End CodeBase review

The application is a React/TypeScript single-page interface built with Vite. Its base URL is /TeacherTraining/, as configured in vite.config.ts. Keep this path when opening local screens or previewing the production build.

Path / file	What it does
src/main.tsx	Loads the initial database/public snapshot before mounting the application and restores current role/session metadata.
src/App.tsx	Chooses public screens, internal pages and role-specific dashboards; applies route access rules and page titles.
src/auth.ts	Defines eight roles, role labels, menu/route access, login OTP completion and logout.
src/api.ts	Encrypted HTTP calls, bearer token/session storage, response handling and request timeouts.
src/dataStore.ts	In-memory operational cache backed by API snapshots; calculates upserts/deletes and row versions, serializes writes and handles unsaved failures.
src/store.ts	Redux state for dashboard audience, selected phase, filters and sidebar state; not the database of operational records.
src/Login.tsx	Password + OTP login and non-demo password-reset UI.
src/Projects.tsx; src/HolidayMaster.tsx	Project/phases/access/association management and holiday/access administration.
src/components/TrainingPlan.tsx; Batches.tsx	Reusable plans, resources, completion criteria, project-linked batches and schedules/import handlers.
src/components/OpenRegistration.tsx; RecoverTraineeId.tsx	Public batch registration with consent/OTP and contact-based trainee-ID recovery.
src/components/RegistrationRequests.tsx; Attendance.tsx	Registration review/approval/import and attendance/amendment/exclusion workflows.
src/components/TrainerRepository.tsx; src/trainerStore.ts	Trainer profiles, skills and availability with backend persistence.
src/components/AssessmentFeedback.tsx	Form configuration, assessment starts/submissions, server-returned results and feedback.

Path / file	What it does
src/components/CertificateTemplates.tsx	Template configuration and current administrator issue/reissue/revoke/print behavior.
src/components/*Dashboard.tsx; DashboardParts.tsx	Role-specific dashboard layouts, navigation/header controls and shared parts.
src/ContentLibrary.tsx; public/content-library-assets	Resource catalog and local source sample assets; backend downloads use signed URLs.
src/reports/; src/data/reportData.ts	Batch performance, training summary, attendance and assessment/feedback reports using server data.
src/data/	School/geography/reference data and report contract helpers; active school data also exists in PostgreSQL.
src/sourceMarkup.ts; public/legacy-dashboard.js	Retained sponsor/reporting dashboard markup and behavior; its operational metrics/trend now come from database aggregates.
src/*.css; public/assets	Existing design, themes, responsive styles and branding; no design restructuring was required.
src/demoSampleData.ts	Development seed source used by the export/seeder tool; it is not run as a browser data fallback.
.env.local (ignored)	Per-machine gateway URL, transport encryption setting and business time zone. Generated/synchronized by Start-Local.ps1.
package.json; package-lock.json; tsconfig.json; vite.config.ts	Scripts, locked dependency graph, TypeScript and /TeacherTraining/ base path.
USER_MANUAL.md; Teacher_Student_Training_Tracker_User_Manual.docx	Supplied user-facing role/menu guides; the Markdown guide includes local OTP notes. No separate user.md was found.
Test_Cases.md; docs/Test_Cases.html	Frontend test-case documentation. These documents do not by themselves prove every case has been executed.
.github/workflows/deploy.yml	Original GitHub Pages static build/deployment workflow; it does not host the .NET API or PostgreSQL.
node_modules/; dist/; .git/	Installed dependencies, generated build output and Git metadata. Reinstall/rebuild rather than copying dependency/build directories.

Business screens use the API adapter, while browser storage retains session information and presentation preferences. The frontend still has an in-memory cache between responses; a successful write is not reported until the API saves it. Date, registration and completion helpers also provide immediate UI feedback, while the server remains authoritative.

The existing hosted GitHub Pages URL was a design/reference deployment. This handover is connected locally. Publishing the current integrated frontend requires a reachable HTTPS API, correct CORS and build-time configuration; a GitHub Pages upload alone will not create a working hosted backend/database. VITE settings are compiled into browser code and should not be treated as private server secrets.

6. Back-End CodeBase review

```
Back-End CodeBase/
  AddingDimensions.sln
  global.json / Directory.Build.props
  Microservices/TeachersAPI/
    Controllers/  Services/  Models/  Utility/  Tests/
  JwtAuthenticationManager/
    Utility/
  OcelotGateWay/
    Program.cs  ocelot.json
  DB Scripts/
    Table Creations/  Stored Procedures/
  Tools/  Tests/
  Start-Local.ps1 / Stop-Local.ps1
  README.md / INTEGRATION-ANALYSIS.md
  .local/
```

6.1 TeachersAPI

TeachersAPI is the one deployed domain API project under Microservices. It currently hosts all training domains; there is not a separate deployable microservice/database per feature. Classes follow the reference pattern of explicit controller/service classes, constructor injection, fields, try/catch, typed parameters and Message<T> responses.

Layer / main files	Responsibilities
Controllers/	Action-based endpoints: accounts, public access, workspace, projects/plans/batches, trainers, registration, attendance, resources, forms, certificates, reports/settings-related collections and legacy Teachers.
Services/*Service.cs and interfaces	Module entry points, business operations and authorization-aware responses. Generated module GetData methods return the caller-authorized workspace snapshot; this is not a paginated public table API.
PortalAccountService; PortalOtpService; PortalPublicService	Hashed-password verification, OTP lifecycle, local outbox, JWT completion, recovery, reset and public registration.
PortalRepository; PortalRecordMapper	Stored-procedure calls, snapshot loading and mapping of existing frontend IDs/contracts to typed relational columns and foreign keys.
PortalBusinessRules	Membership/scope-sensitive validation, dates, schedules, attempt rules, assessment scoring and certificate completion criteria.
PortalService; ResourceFileService	Collection save/read orchestration, contact masking, answer-key removal for trainees and signed local file delivery.
Models/	EncryptedRequest, Message<T>, request validation and portal/legacy teacher models.
Utility/	AES-GCM codec, response/request/error helpers, PII protection and repeatable local demo seeding.
Program.cs	Dependency injection, config loading, controller validation, rate limiting, exception handling, Swagger and development-only seed modes.
appsettings.json; appsettings.Local.json	Committed non-secret defaults and ignored per-machine database/JWT/encryption/CORS overrides. Local files are excluded from publish/output copying.
Tests/	Original isolated Teachers API smoke test and independent encryption test client.

6.2 JwtAuthenticationManager

The shared library configures JWT signature/issuer/audience/expiry checks, distinct portal and legacy identity policies, database session validation and token issuance. Portal identities are app_user records; legacy Teacher identities are separated so equal numeric IDs cannot grant cross-application access. SqlHelper wraps Npgsql connections/commands and uses actual stored-procedure execution, not inline SQL text.

6.3 OcelotGateWay

Ocelot is the browser-facing API entry point. ocelot.json forwards public account/public/health routes and protected training routes to TeachersAPI. A signed-resource route is separately configured. The gateway applies JWT/session validation, local CORS and basic response/security headers and records request status/timing. It needs the same database/session and JWT configuration as TeachersAPI. Configured downstream port is 5101.

6.4 SQL folders, helpers and generated files

Path	Meaning / correct use
DB Scripts/Table Creations	Ordered workbook table DDL, indexes/keys, BRD extension tables, assessment/certificate rules, identity-link repair and six views.

Path	Meaning / correct use
DB Scripts/Stored Procedures	All named application procedures plus access/scoring/helper functions. PostgreSQL procedures contain the actual SQL operations.
DB Scripts/workbook-schema-manifest.json	Source column/type/key/reference inventory extracted from the client workbook. It distinguishes the workbook foundation from later extensions.
DB Scripts/GenerateWorkbookSchema.py	Recreates source workbook DDL for review; do not casually regenerate it over curated extension work.
DB Scripts/ConfigureLocalDatabase.py; CreateApplicationDatabase.sql; ConfigureApplicationOwnership.sql	Earlier low-level local bootstrap/ownership helpers. The complete snapshot handover should use Database Import/setup_local.py instead. Do not mix bootstrap methods.
Tools/DeployLocalDatabase.py; DB Scripts/DeployDatabase.ps1	Apply ordered table/procedure definitions. Snapshot recipients should restore first and do not need to deploy the same schema again.
Tools/ExportFrontendDemo.cjs; Utility/PortalDemoSeeder.cs	Export deterministic demo inputs and fill missing local sample records without replacing existing demo passwords. A restore already contains samples; seeding is not required for normal import.
Tools/GeneratePortalCollections.py; ConnectFrontend.cjs; ConnectDashboardData.py	Implementation/migration tooling. Connected frontend migration helpers detect an already-applied migration; these are not daily launch commands.
Tools/ExportPortableDatabase.py; PortableSetup.py	Generate the handover and maintain the setup helper copied as Database Import/setup_local.py.
Tools/RunLocalSql.py; DB Scripts/VerifyLocalData.sql	Explicit maintenance/diagnostic SQL execution using ignored local application credentials.
DB Scripts/CleanupIntegrationFixtures.sql	Manual/test-runner cleanup of reserved TEST-* fixtures by soft deletion; not part of normal schema deployment.
Tests/	Encrypted API checks, OTP tests, UI workflows/navigation, real attendance checks and isolated portable-restore verification.
.local/	Ignored runtime resources, outbox, process tracking, logs, test evidence and tool environments. Only required learning files are exported in the package.
bin/; obj/; .vs/	Generated build and IDE state; recreate on the receiving machine.

7. PostgreSQL design and sample data

The export contains 64 public tables, 6 views and 31 stored procedures, plus helper/trigger functions, identities, keys, indexes and application data. The workbook defines 49 domain tables and 560 original columns. The remaining tables cover two starter entities and integration/BRD additions; additional compatibility/audit columns on workbook tables are intentional.

Domain	Representative entities
Identity/access/privacy	app_user, m_role, role_permission, user_access_scope, app_auth_session, consent_record, privacy metadata/logs
Geography/schools	m_state, m_district, m_block, m_cluster, m_school, m_holiday
Projects and plans	project, project_phase, training_plan, training_plan_day/activity, project_plan_association, Odoo staging catalog
Training delivery	trainer, trainer_availability, training_batch, batch_day/activity, trainee, batch_enrollment
Learning and assessment	learning_resource, form_question, form_submission, assessment_attempt, assessment_start, resource_progress/access logs
Completion/certificates	completion_review, certificate_template, certificate_record, external_course_record

Domain	Representative entities
Integration/operations	portal_collection, portal_setting, portal_otp_challenge, portal_recovery_request, school_eligibility, audit/activity logs

Typed relational keys, dates, statuses, scopes and operational values are populated alongside external_id and ui_data compatibility fields. ui_data retains screen-specific fields; it does not replace the entire database with one browser-state document. Row versions prevent stale updates, and soft-deleted test/history rows can remain in the development snapshot.

Active source-snapshot records	Count
plans	5
batches	2
schools	750
projects	1
trainees	10
resources	5
attendance	3
registrations	11

The base samples include the eight roles, 750 schools, one project with phases, past/upcoming batches, a trainer/trainees, saved attendance, resources/forms and template examples. Extra sample plans and trainee accounts created during verification may also be present. Only the eight stable Demo credentials below are documented; credentials of newly registered accounts are delivered through their own local email, not invented here.

The custom archive and plain SQL represent the same dump snapshot. They omit row data from app_auth_session, teacher_auth_tokens, portal_otp_challenge, portal_recovery_request and assessment_start. Their schemas are retained. This excludes active logins, one-time codes and in-progress assessment starts without deleting historical submitted results. The setup helper clears imported login lock counters before use. Database/server roles and source server passwords are not exported.

IMPORTANT FOR PORTABILITY: keep application-encryption.private.json with the database package. Generating different PII keys would make existing encrypted contacts unreadable and change blind-index matching. The package contains development data/password hashes and private encryption material; transfer it privately. The setup helper generates a new database password, JWT signing key and AES transport key for each destination. The Word/PDF do not contain these private values.

8. Reference Documents review

Original reference	Purpose and implementation relationship
design thinking Db and partition.xlsx	Primary database design/data dictionary: table definitions, source column types, identities, keys, references and reporting/access views. Used by GenerateWorkbookSchema.py; source workbook is retained unchanged.
Training-Tracker-Business-Requirements v2.docx	Business functional requirements, role expectations, workflow rules and revised decisions. Guides registration, scheduling, attendance, assessments, completion and reporting; outstanding acceptance gaps are explicitly identified in section 16.
Queries and responses.docx	Client clarifications and responses that provide context for design decisions and deferred/external integrations.
Adding Dimensions Dashboard Screen.png	Reference for the sponsor/phasewise dashboard layout, KPI/attendance/batch panels and location reporting. Current numerical values come from the local database rather than the image.

Keep Reference Documents with the handover for future changes, but the application does not open Word/Excel during a request. Earlier NISM Sync MicroServices sources were used as a coding/encryption/style reference; copying that older .NET 8 / SQL Server project is not required to run this .NET 10 / PostgreSQL application.

9. Demo credentials and role access

User ID	Password	Role	Typical access
Demo1	demo1@123	Sponsor Admin	Phasewise/sponsor dashboard; scoped projects, certificate administration and MIS reports.
Demo2	demo1@123	Schoolnet Super Admin	Schoolnet dashboard; broad local project, plan, batch, attendance, trainer, forms, templates, holiday/user/access management.
Demo3	demo1@123	Schoolnet Project Admin	Schoolnet dashboard and assigned-project operations; no super-admin-only holiday/user administration.
Demo4	demo1@123	State Admin	State dashboard, assigned geography/projects/batches, registration and allowed reporting/operations.
Demo5	demo1@123	School Admin	School dashboard and permitted school/project/batch/registration views and operations.
Demo6	demo1@123	Trainer	Assigned training batches, own trainer profile/availability, registration review, attendance and learning/reporting access.
Demo7	demo1@123	Trainee	Personal dashboard, own registrations/learning/assessment/feedback/completion information.
Demo8	demo1@123	Reporting/Audit User	Phasewise/reporting dashboard; scoped project/report viewing, no operational administration.

These are application accounts, not PostgreSQL server accounts. The original shared demo password is intentionally unchanged for all eight IDs. Seeding does not overwrite existing passwords, and password reset rejects changes to Demo1-Demo8. Menu visibility is not the sole permission check: frontend route rules and backend role/scope/ownership checks apply. Some menus route to personal dashboard records instead of the administrator certificate screen.

10. Login, OTP, registration and recovery flows

10.1 Demo login

- Open http://localhost:5173/TeacherTraining/login. Enter a Demo ID and demo1@123, then choose Sign in.
- The server verifies the stored password hash and account status, then creates an OTP challenge. A password alone does not issue a JWT.
- Choose Open local email outbox. Read the six-digit code from the browser text message, return to the login tab and choose Verify and sign in.
- The verified JWT/session loads the role-authorized workspace and dashboard from PostgreSQL. Sign out from the header to revoke it.
- One active portal session per account is retained: a later login to the same ID invalidates the earlier session. Use different demo IDs for simultaneous role reviews.

10.2 OTP rules and local delivery

Codes expire after five minutes and can be used once. Three incorrect attempts lock verification for the contact for 15 minutes. Login/public request throttles also apply. Challenges store a keyed OTP hash and encrypted bound payload; code values are not included in normal API JSON. Local messages are .eml files under Back-End CodeBase/.local/mail-outbox. The request-specific browser viewer requires Development and a loopback connection. This is local testing, not a real

mailbox integration. Outside Development, requests require a configured delivery provider.

10.3 Open registration

- Open /TeacherTraining/register?batch=BATCH-DEMO-NOV, or use the Open registration link and enter the shared batch code.
- Review batch details/window/capacity. Enter full name, email or mobile, participant category and consent. Request the verification code.
- Read the local outbox message, enter the code and choose Verify and submit registration. Verified details are bound to the challenge; they cannot be replaced by a modified final request.
- The server validates the real registration window/capacity and contact/duplicate rules, then creates or reuses the trainee identity and inserts a pending enrollment/consent/audit record.
- A newly created account receives its initial credentials through a separate local email link. Registering an existing account does not replace that account password.
- An authorized trainer/administrator reviews the request on Registration Requests. The trainee sees only their own linked records.

BATCH-DEMO-NOV starts 10-Nov-2026 and has a 14-day pre-registration window: 27-Oct through 10-Nov-2026. It will be correctly closed outside that window on another date. An authorized administrator must update or create a batch with valid future dates/window and compatible trainer/school/plan data. Do not disable database window/attendance validation to make old samples appear current.

10.4 Recovery and password reset

Forgot your Trainee ID? starts an OTP challenge for a registered contact and returns the trainee ID only after verification. Forgot password? verifies an account/contact with OTP, then accepts a new non-demo password of at least ten characters and revokes existing sessions. Demo account reset is blocked to retain the documented credentials. Neither flow sends external mail during local testing.

11. New-machine setup: recommended method

11.1 Install prerequisites and copy files

- Install the .NET SDK 10.0.403, Node.js 24 LTS, PostgreSQL 18 with command-line tools, Python 3.11+ and a browser. pgAdmin is optional; a running PostgreSQL server is required.
- During PostgreSQL installation, set and remember YOUR new machine administrator password. Do not reuse an assumed password from the source developer PC.
- Copy Front-End CodeBase, Back-End CodeBase, Reference Documents and Database Import into one parent folder. If using the ZIP, extract it into the parent so Database Import/setup_local.py exists.
- Do not transfer source-machine appsettings.Local.json, frontend .env.local, .local process/log/outbox state, node_modules, bin, obj or .vs as setup inputs. Copy source/assets/lockfiles and the supplied import package. The package separately includes required learning files.
- Ensure ports 5432 (or your selected PostgreSQL port), 5100, 5101 and 5173 are available. The launcher binds the application locally.

```
$ProjectRoot = 'C:\Projects\Adding Dimensions'
Set-Location $ProjectRoot
# If your local script policy requires it, for this PowerShell session only:
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
python --version
dotnet --version
node --version
npm.cmd --version
```

11.2 Restore and configure in one command

```
python '.\Database Import\setup_local.py' `
  --backend '.\Back-End CodeBase' `
  --postgres-bin 'C:\Program Files\PostgreSQL\18\bin'
```

The script prompts securely for your local PostgreSQL administrator password. It checks package SHA-256 hashes, refuses existing destination configuration/database/role names, creates adding_dimensions and the non-superuser adding_dimensions_app role, restores the archive, verifies schema/readiness, copies resources and writes matching ignored local settings for API/gateway. It does not save the administrator password. No application password has to be typed into committed source files.

If the default database or role already exists, use fresh names rather than dropping someone else's database:

```
python '.\Database Import\setup_local.py' `
  --backend '.\Back-End CodeBase' `
  --database adding_dimensions_demo `
  --app-user adding_dimensions_demo_app `
  --host localhost --port 5432
```

An existing appsettings.Local.json is a separate stop condition: use a fresh checkout/folder or deliberately preserve/move your own previous configuration before retrying. The helper never replaces it. If a restore fails after creating fresh objects, those new objects may remain for inspection; the script does not automatically drop databases. Correct the issue and use a new target name or complete the manual configure-only workflow.

11.3 Install/build frontend and backend

```
Set-Location (Join-Path $ProjectRoot 'Front-End CodeBase')
npm.cmd ci
npm.cmd run build

Set-Location (Join-Path $ProjectRoot 'Back-End CodeBase')
dotnet restore .\AddingDimensions.sln
dotnet build .\AddingDimensions.sln -c Release --nologo
.\Start-Local.ps1 -SkipBuild
```

Start-Local.ps1 synchronizes the ignored frontend .env.local from the new API transport configuration, runs TeachersAPI/gateway/Vite in hidden processes, waits for readiness and prints the local URLs. Keep the two codebase folders as siblings. You can use Start-Local.ps1 without -SkipBuild for later startup/builds; stop tracked running services before rebuilding.

11.4 Open and verify

Check	Address / command
Frontend login	http://localhost:5173/TeacherTraining/login
Open registration	http://localhost:5173/TeacherTraining/register?batch=BATCH-DEMO-NOV
Gateway readiness	http://localhost:5100/api/Health/ready
API readiness	http://localhost:5101/api/Health/ready
Local Swagger	http://localhost:5101/swagger

```
Invoke-RestMethod 'http://localhost:5100/api/Health/ready'
# Expected: isSuccess True, statusCode 200, returnMessage Healthy.
# Open the login URL manually in Chrome and use the demo table + OTP outbox.
# Stop only this project's tracked services when finished:
.\Stop-Local.ps1
```

A successful handover means: readiness is healthy; Demo2 completes password/OTP login; schools/projects/batches display; an authorized save survives reload; Demo7 sees own records; a learning-file link downloads; public registration works for a currently open batch. The imported database already includes sample data, so no seed command is required.

12. Manual PostgreSQL import and pgAdmin guidance

Use one restore format, not both on the same database. The custom archive is recommended. Both contain schema and data but no PostgreSQL global login roles or source database password. Create a fresh non-superuser application login and an empty UTF-8 database owned by it first, using pgAdmin or psql as your destination administrator. Run restores as that application role so objects belong to it.

12.1 Custom archive with pg_restore

```
$PgBin = 'C:\Program Files\PostgreSQL\18\bin'
# First create adding_dimensions_app and the empty database on this machine.
# -W prompts for the DESTINATION application's PostgreSQL password.
& "$PgBin\pg_restore.exe" -h localhost -p 5432 `
  -U adding_dimensions_app -W -d adding_dimensions `
  --no-owner --no-privileges --exit-on-error --single-transaction `
  '.\Database Import\AddingDimensions.backup'
```

12.2 Plain SQL with psql

```
& "$PgBin\psql.exe" -X -h localhost -p 5432 `
  -U adding_dimensions_app -W -d adding_dimensions `
  -v ON_ERROR_STOP=1 --single-transaction `
  -f '.\Database Import\AddingDimensions.sql'
```

Use PostgreSQL 18 psql for this SQL file: it contains pg_dump/pg_restore-generated psql directives as well as SQL. Import it with psql, not by blindly pasting the whole file into pgAdmin Query Tool. Do not select the .sql file as a custom-format pgAdmin Restore archive.

12.3 pgAdmin route

- Connect to your local PostgreSQL server with the administrator account; this is independent of the Demo application accounts.
- Under Login/Group Roles create a login-capable application role with a new local password and no superuser/role/database-creation privileges.
- Create a fresh UTF-8 database with the application role as owner. Connect with that role for restore, or set the restore role appropriately.
- Right-click the empty database, choose Restore and select AddingDimensions.backup (custom format). Enable the equivalents of No owner and No privileges so original source ownership/grants are not required. pgAdmin option locations vary by version.
- Inspect the restore process log; success must be checked, not inferred from the dialog closing. Refresh Schemas > public > Tables/Views and confirm the object inventory.
- Use configure-only below to copy the matching PII keys/resources and generate new local app secrets. Restoring database objects alone is not a complete application setup.

12.4 Configure an already-restored database

```
python '.\Database Import\setup_local.py' `
  --configure-only --backend '.\Back-End CodeBase' `
  --database adding_dimensions --app-user adding_dimensions_app `
  --host localhost --port 5432
```

Configure-only prompts for the destination application role password, verifies the restored database, copies learning files and writes the two local configurations using the matching exported PII keys. It still refuses existing settings/resources. Then continue with npm ci, .NET build and Start-Local.ps1. Do not regenerate PII keys for the restored snapshot.

13. Daily running, configuration and development

Setting / artifact	How it is used
API/gateway appsettings.Local.json	ConnectionStrings:DBConnection, shared Jwt:SecurityKey, AESPassword, Pii keys, local CORS, business time zone and gateway resource URL. Files are ignored and excluded from publish copying.
Frontend .env.local	VITE_API_BASE_URL=http://localhost:5100, matching VITE_AES_PASSWORD and VITE_BUSINESS_TIME_ZONE=Asia/Kolkata; synchronized at startup.
ASPNETCORE_ENVIRONMENT	Development is required for the selected local OTP outbox and Swagger; launch script sets it for its child services.
Default ports	Frontend 5173, gateway 5100, API 5101, PostgreSQL 5432. Changing application ports also requires launcher, Ocelot downstream, CORS, frontend API base and signed-resource gateway URL changes.
.local/processes.json; Stop-Local.ps1	Tracks only this project's API/gateway/frontend processes. Stop before rebuilding; do not stop unrelated database/services.
.local/*.out.log / *.err.log	Service startup and request/error diagnostics. Secrets/config contents should not be pasted into public reports.
.local/mail-outbox	Local test OTP/account mail only; it is not copied into the handover. New requests create fresh messages on the receiving machine.
.local/resources	Runtime learning files; must accompany database metadata. Included by the import helper.
Existing school/resource IDs and dates	Seeded samples are fixtures, not a guarantee that every batch remains currently registerable or every user satisfies certificate criteria.

For normal startup use Start-Local.ps1; to stop use Stop-Local.ps1. Do not run database bootstrap/seed scripts every time the application starts. Apply reviewed schema/procedure updates with the deployment helpers when source SQL intentionally changes. Use npm run build and dotnet build for validation. The current frontend build succeeds with an existing bundle-size advisory; this is not a database/authentication failure.

14. Validation and test evidence

Verification	Recorded result / limits
Backend build	Release build passed with zero warnings/errors in the local implementation.
Frontend build	TypeScript + Vite production build passed. Existing CSS bundle remained unchanged during integration.
PortalIntegration.cjs	74 encrypted API/database checks across eight demo roles, including denial/scope/version checks, attendance, timed assessment scores and certificate issue/reissue/revocation.
PortalOtp.cjs	26 checks for OTP, registration/approval, email login, trainee ownership, recovery/reset, session invalidation and preserving demo passwords.
PortalFlows.cjs	13 Chrome UI checks: login, navigation, plan save/reload, registration/OTP, new trainee login and password reset.
PortalBrowser.cjs	44 screens across eight roles, 84 real API responses, with no recorded page errors/failed API calls in that run.
PortalDashboard.cjs	6 targeted Chrome checks comparing daily attendance/KPI values to database aggregates and checking no-data audience switching.
Original Teachers smoke suite	125 checks against an isolated temporary PostgreSQL instance for stored procedures, AES compatibility, TeachersAPI and Ocelot.
Portable import verification	Both custom archive and SQL imported into fresh databases in an isolated PostgreSQL cluster on this machine. The restored API verified all eight original demo passwords with OTP, decrypted existing contacts and downloaded a signed resource. Active session/OTP/start data were absent.

These results are evidence for the current local build, not proof of every case in the supplied test catalog, every role/record combination, another operating system or production BRD acceptance. Source results are in Back-End CodeBase/.local, and the package includes its portable restore-verification summary.

```
# Optional browser test tooling on a fresh machine:
Set-Location (Join-Path $ProjectRoot 'Back-End CodeBase')
npm.cmd install --prefix .local/browser-runner playwright
# Google Chrome must be installed; suites use its Playwright channel.
.\Tests\RunLocalChecks.ps1
# Separately, with PostgreSQL server utilities installed:
.\Microservices\TeachersAPI\Tests\SmokeTest.ps1
```

The integration runner deliberately waits between suites for login-rate-limit windows and creates/soft-deletes reserved TEST-* fixtures. UI-created sample plans/registrations can remain for review. Fixed-date API fixtures are based on the October 2026 sample period and must be reviewed before rerunning the suite on a later date; do not mistake a date-rule rejection for a broken restore. Diagnostic test tooling is optional for ordinary application use.

15. Troubleshooting

Symptom	Check / fix
SDK not found or global.json mismatch	Install .NET SDK 10.0.403 / compatible 10.0.4xx patch; verify dotnet --list-sdks in the backend directory. A runtime-only or 10.0.1xx installation is insufficient for this pin.
psql/pg_restore not found	Install PostgreSQL 18 command-line tools or pass --postgres-bin with the actual installation path. A pgAdmin-only installation is not the server/toolchain.
Database/role already exists	The importer refuses overwrites. Use a new database and role name, or manually restore into a deliberately prepared empty database and run configure-only.
Local configuration already exists	Use a fresh source copy and exclude the old machine's appsettings.Local.json/.env.local; preserve any existing local work intentionally.
PostgreSQL authentication failure	Use the receiving machine's admin password for provisioning and application-role password for manual restore/configure-only. Verify host, port, user and service status.
Checksum mismatch or contacts fail to decrypt	Re-extract/copy the complete package. Use the matching private PII file; do not generate replacement keys for existing encrypted rows.
Frontend packages missing	Run npm.cmd ci inside Front-End CodeBase; do not copy another machine's node_modules.
Port already in use / DLL locked during build	Run Stop-Local.ps1 for this project before rebuilding. Use Get-NetTCPConnection to identify unrelated listeners instead of terminating arbitrary processes.
CORS/network/encryption error	Start all services; verify gateway URL 5100, API downstream 5101, CORS origin 5173 and matching AES configuration. Restart after changing .env.local.
Login succeeds but a prior tab signs out	The account has a newer active session or expired JWT. Use different Demo IDs for simultaneous sessions and sign in again with a fresh OTP.
OTP invalid / 429 / verification lock	Codes are single-use/five-minute. Request a fresh code when allowed, wait for throttling windows, and respect the 15-minute lock after three incorrect attempts. Do not publish local outbox URLs.
Registration is closed	Check the real machine date against the seeded batch window. Edit/create a valid batch as Demo2 rather than bypassing server rules.
Missing resources or expired links	Ensure imported resources were copied under backend .local/resources. Sign in again and reload to obtain a fresh signed URL.
Empty dashboard/permission denial	Check role and assigned scope, approval/ownership, active filters and database records. Empty collections intentionally show no data rather than fake figures.
GitHub Pages deployment cannot log in	Static hosting does not provide API/database access. Configure a real HTTPS backend and build-time endpoint/settings before publishing an integrated remote frontend.

16. Remaining work and deployment boundaries

- Real email/SMS OTP delivery is not configured. Local mail and viewer are Development/loopback only; production authentication needs a provider and deployment configuration.
- Odoo selection uses local staging/examples. A live Odoo synchronization connector and automated external-course verification are not configured.
- Certificate issuance currently follows the existing administrator screen. Separate trainer-final approval and its corresponding UI/workflow require additional work/acceptance.
- Advanced configurable privacy exceptions/administration, full data-subject processes and complete BRD acceptance remain pending; the presence of workbook metadata tables does not mean every administrative feature is implemented.
- The frontend and .NET solution build locally, but production HTTPS hosting, operational key management, deployment automation for backend/database, monitoring, recovery/backup schedules and performance/load acceptance are not provided by this export.
- The automated launcher and documented handover were verified on Windows. Cross-platform runtime support does not imply the Windows PowerShell launch scripts work unchanged on Linux/macOS.
- This import is a development/demo snapshot, not a sanitized production starter. Stable Demo passwords are intentionally documented as requested and must not be treated as production credentials.

Appendix A. Database table inventory

Public schema objects in the exported snapshot. Column counts include intentional integration extensions; these counts are not the original 560-column workbook count.

Table	Columns
app_auth_session	4
app_user	33
assessment_attempt	17
assessment_start	15
attendance	17
attendance_exclusion	14
audit_log	18
batch_day	14
batch_day_activity	17
batch_enrollment	29
batch_registration_link	23
bulk_upload_log	20
certificate_record	20
certificate_template	22
completion_review	16
consent_record	18
data_subject_request	18
external_course_record	12
form_question	20
form_submission	15
learning_resource	29
m_block	17
m_cluster	18
m_district	17
m_holiday	20
m_module	17
m_pii_field	19
m_role	20
m_school	21

Table	Columns
m_stakeholder	18
m_state	16
odoo_project_catalog	12
pii_access_log	16
plan_resource_day	11
plan_resource_link	18
portal_collection	5
portal_otp_challenge	19
portal_recovery_request	14
portal_setting	10
profile_change_log	14
project	21
project_phase	15
project_plan_association	12
registration_otp	22
resource_access_log	12
resource_audience	17
resource_progress	16
role_creation_rule	16
role_permission	22
role_pii_permission	18
school_eligibility	13
system_setting	23
teacher_auth_tokens	3
teachers	11
trainee	25
trainer	24
trainer_availability	17
training_batch	29
training_plan	21
training_plan_activity	22
training_plan_day	18

Table	Columns
user_access_scope	23
user_activity_log	15
user_pii_override	20

Appendix B. Views and stored procedures

View	Purpose
v_batch_progress	Batch enrollment, attendance, completion and certificate aggregates.
v_batch_schedule	Batch day/activity schedule.
v_project_status	Project-level training status.
v_school_geography	School/geographic joins.
v_trainee_resource_access	Trainee learning-resource access/reporting.
v_trainer_assignment_history	Trainer batch assignment history.

Stored procedure	Family
usp_check_database_health	Legacy Teachers / health
usp_delete_teacher	Legacy Teachers / health
usp_get_login_user	Legacy Teachers / health
usp_get_teacher_by_id	Legacy Teachers / health
usp_get_teachers_list	Legacy Teachers / health
usp_get_user_auth_token	Legacy Teachers / health
usp_portal_apply	Portal application
usp_portal_begin_assessment	Portal application
usp_portal_bootstrap	Portal application
usp_portal_check_session	Portal application
usp_portal_create_otp	Portal application
usp_portal_dashboard	Portal application
usp_portal_login_lookup	Portal application
usp_portal_login_result	Portal application
usp_portal_public_data	Portal application
usp_portal_public_register	Portal application
usp_portal_recovery_lookup	Portal application
usp_portal_request_recovery	Portal application
usp_portal_reset_password	Portal application

Stored procedure	Family
usp_portal_resource_access	Portal application
usp_portal_revoke_session	Portal application
usp_portal_save_session	Portal application
usp_portal_seed_scopes	Portal application
usp_portal_snapshot	Portal application
usp_portal_sync_children	Portal application
usp_portal_verify_otp	Portal application
usp_portal_verify_recovery	Portal application
usp_revoke_auth_token	Legacy Teachers / health
usp_save_auth_token	Legacy Teachers / health
usp_save_teacher	Legacy Teachers / health
usp_update_teacher	Legacy Teachers / health

Helper/trigger functions in the export: portal_audience_metrics, portal_project_access, portal_row_access, trg_set_updated_at, trg_validate_assessment_attempt. The two SQL folders retain readable procedure/function definitions for maintainers. The archive also carries indexes, identity sequence values and foreign keys so new inserted records do not reuse old identities.

Appendix C. Frontend collections and API routes

Typical module operations are POST /api/{Controller}/GetData and POST /api/{Controller}/SaveData. Requests use the encrypted envelope and a bearer token. GetData exposes the caller-authorized workspace rather than a single public table. Write-role and row-scope checks are enforced in services/procedures; naming a route does not grant permission.

Frontend collection suffix (teacher-training-...)	Controller
projects	Projects
plans	TrainingPlans
project-plan-associations	ProjectPlans
batch-plan-uses	Batches
trainer-profiles	Trainers
trainer-availability	TrainerAvailability
trainees	Trainees
registrations	Registrations
attendance-records	Attendance
attendance-exclusions	AttendanceExclusions
assessment-form-master	AssessmentForms
assessment-records	Assessments

Frontend collection suffix (teacher-training-...)	Controller
feedback-records	Feedback
certificate-templates	CertificateTemplates
certificates	Certificates
holidays	Holidays
users	Users
odoo-project-catalog	OdooProjects
schools	Schools
learning-resources	Resources
resource-usage	ResourceProgress
external-course-records	ExternalCourses
eligible-teachers	EligibleTeachers
eligible-students	EligibleStudents
audit	Audit
trainer-audit	TrainerAudit
certificate-audit	CertificateAudit
school-assignments	Settings
school-access	Settings
school-admin-assignments	Settings
state-access	Settings
cluster-master	Settings
approved-venues	Settings
school-eligibility	Settings
eligible-participants	Settings
holiday-policy	Settings
batch-dependent-records	Settings

Special route	Purpose
POST /api/Account/Login	Verify password and create OTP challenge.
POST /api/Account/VerifyLogin	Verify OTP, issue JWT/session.
POST /api/Account/Logout	Revoke current session.
POST /api/Account/RequestPasswordReset; ResetPassword	Non-demo password reset with OTP.
POST /api/Public/GetData	Public registration reference data without the authenticated workspace.

Special route	Purpose
POST /api/Public/RequestRegistration; Register	Request registration verification, then atomically submit verified details.
POST /api/Public/RequestRecovery; VerifyRecovery	Contact-based trainee-ID recovery.
GET /api/Public/LocalOutbox/{id}	Development-only loopback outbox message.
POST /api/Workspace/GetData	Current role-authorized database snapshot.
POST /api/Assessments/Begin	Server-timed assessment start with membership/availability checks.
GET /api/Resources/Download?ticket=...	Signed file download; session and scope checked.
POST /api/Resources/Upload	Authorized multipart file upload.
GET /api/Health/ready	Database/application readiness.
/api/Login/...; /api/Teachers/...	Retained legacy teacher starter endpoints; separate identity policy from portal users.

Appendix D. Source and download references

Project facts come from the inspected source folders, reference files, package manifests, database export inventory and recorded test outputs. Versions above describe the tested project, not a recommendation to replace the lockfiles with whatever is newest.

- Frontend source repository: https://github.com/VSKSurya/TeacherTraining
- Existing hosted frontend/design reference: https://vsksurya.github.io/TeacherTraining/login
- .NET 10 SDK downloads: https://dotnet.microsoft.com/en-us/download/dotnet/10.0
- Node.js downloads: https://nodejs.org/en/download
- PostgreSQL Windows installer: https://www.postgresql.org/download/windows/
- PostgreSQL 18 pg_dump reference: https://www.postgresql.org/docs/18/app-pgdump.html
- PostgreSQL 18 pg_restore reference: https://www.postgresql.org/docs/18/app-pgrestore.html

The export/setup helpers were created for this handover and verified against isolated databases. Keep the document with the import package and the three major source/reference folders. Review the manifest creation time when determining which snapshot a recipient is importing.